For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings for the representative prior-work benchmarks. We remark, however, that Sppl supports arithmetic, as well as (soft) conditioning on measure zero events, which Slice does not support. Our evaluation merely serves the purpose of investigating (i) the overhead of discretization and (ii) the benefits of being able to use efficient *BDD-based* inference back-ends (i.e., Dice and Roulette) for exact inference on *continuous programs*.

7.1 Slice+Storm: Case Studies

There is no exact inference system that, to our knowledge, can cover programs combining all three features: continuous sampling, higher-order functions, and unbounded recursion. As a consequence, there is no exact inference engine for the programs in Sections 2.6, 7.1.2 and 7.1.3. Instead, we use *probabilistic model checking*: The small-step semantics of our language (cf. Tables 1 and 2) for discrete programs gives rise to a *Markov chain* modeling the stochastic process of program execution, inspired by [2] on imperative programs. If that Markov chain is finite, we can use the probabilistic model checker Storm [16] for exact inference. Let us start with an illustrative example.

7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. We generate the corresponding Markov chain by unfolding the small-step semantics in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. If, however, it terminates, the resulting Markov chain is finite and precisely captures the execution behavior of the discretized program. For this example, this is the case: Even though the discretized program contains unbounded recursion, its execution behavior can be modelled as a finite-state Markov chain[2], where $\bot$ is a special state entered when encountered an observation failure.

Now, using Using the probabilistic model checker Storm, we can perform exact inference: It is immediate that the Markov chain's probability $\Pr(\Diamond \texttt{true} \mid \Box\neg\bot)$ of eventually reaching compute three distinct quantities. First, the raw extended semantics records the mass of returning true(in LTL: $\Diamond$true) *given* that we never enter the observation-failure state , the mass of returning false, and the mass of observation failure $\bot$(in LTL: $\Box\neg\bot$) equals the measure of ; its missing mass is divergence. Second, conditioning on observation success divides by $1 - \Pr(\Diamond\bot)$. This retains divergent paths and is therefore not, in general, a distribution over returned values. Third, the normalized posterior over returned values divides by the probability of eventual successful termination:

$$\Pr_{\text{post}}(\text{true}) = \frac{\Pr(\Diamond\text{true})}{\Pr(\Diamond\text{true}) + \Pr(\Diamond\text{false})},$$

provided that the denominator is positive.

The distinction matters for this example. Resolving the 0.2 resampling self-loop gives raw masses $\Pr(\Diamond\text{true}) = 0.375$ and $\Pr(\Diamond\bot) = 0.25$, and divergence mass 0.375. Thus conditioning only on avoidance of $\bot$ gives $\Pr(\Diamond\text{true} \mid \Box\neg\bot) = 0.375/0.75 = 0.5$, because $\Box\neg\bot$ includes the divergent executions. The program has no false-returning execution, so its normalized posterior over returned values instead assigns probability $0.375/0.375 = 1$ to truein the program's normalized posterior distribution.

[2]We apply a straightforward optimization: Intermediate execution steps that do not branch probabilistically can be collapsed into a single transition. This is the reason why all states in Figure 15 correspond to the header of the recursive function.

```
let x = if flip() then 0.5 else 1.5 in gaussian(0, x) < 0.5
```

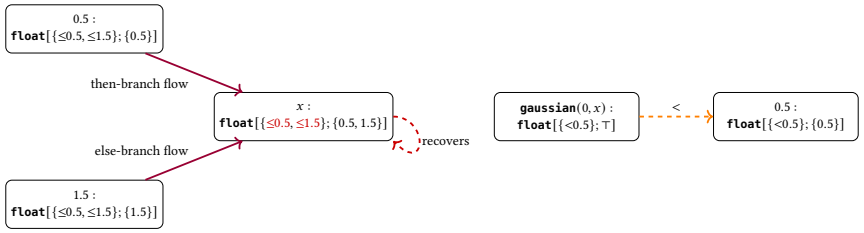


Fig. 19. Constraint graph populated with float type information. As before, solid arrows represent data flow, and dashed arrows represent comparison constraints. Using x as a Gaussian parameter induces a *recovers* constraint, the red dashed self-loop, which ensures a unique discrete index for each value of x.

```
      else
        if perfect <= 0 then
          (4.0 : float[{<10,<=10}; {4}])
        else
          (uniform(0, 4) : float[{<10,<=10}; T])
      : float[{<10,<=10}; T] in
let _ = observe(gpa : float[{<10,<=10}; T] <= 10 : float[{<10,<=10}; {10}]
          && gpa : float[{<10,<=10}; T] >= 10 : float[{<10,<=10}; {10}]) in
nationality > 0
```

The type system propagates the cuts <10 and ≤10 for all GPA expressions. These cuts induce the three regions:

$$(-\infty, 10), \qquad \{10\}, \qquad (10, \infty).$$

Under this partition, both continuous branches, `uniform(0,10)` and `uniform(0,4)`, place all of their probability mass in the first region. The constant 10.0 is mapped to the singleton region $\{10\}$, while the constant 4.0 is mapped to the region $(-\infty, 10)$. The resulting discretized program is therefore:

```
let nationality = discrete(0.5, 0.5) in
let perfect = discrete(0.01, 0.99) in
let gpa = if nationality <= 0 then
        if perfect <= 0 then 1
        else
          discrete(1.0, 0.0, 0.0)
      else
        if perfect <= 0 then 0
        else
          discrete(1.0, 0.0, 0.0) in
let _ = observe(gpa <= 1 && gpa >= 1)
nationality > 0
```

After discretization, the equality test `gpa = 10` becomes the discrete test that the GPA index is exactly the one corresponding to the singleton region $\{10\}$.

A.4 Point and Measure-Zero Observations

Slice can condition on an exact value when that event has strictly positive probability mass, as in the Indian GPA example in §A.3. In that case, conditioning returned-value normalization is well-defined and Slice returns the correct answerpreserves the raw extended semantics from which the posterior is computed. The exact set of programs supported by Slice is therefore somewhat larger than one would initially expect, while not covering full point observations on continuous samples.

The mechanism is the same as in §A.3: the cuts $<c$ and $\leq c$ induce the partition

$$(-\infty, c), \qquad \{c\}, \qquad (c, \infty),$$

so any probability mass assigned exactly to c is tracked as its own discrete outcome.

More precisely:

(1) Slice cannot handle programs where every control-flow path conditions on an event of measure zero, and therefore programs with exact observations on continuous samples are in general out of scope, i.e. the programs one typically writes in e.g. Stan are out of scope unless one widens exact conditioning to a small but finite interval.
(2) Slice can handle programs where at least one control-flow path conditions only on strictly-positive-probability events, even if that is a point observation, and even if other paths condition only on measure-zero events.
(3) It does not matter whether an event is measure-zero because it is point-conditioning on a continuous sample, or whether it is measure-zero for another reason, e.g. **observe**(**uniform**$(0,1) > 3$) or simply **observe**(false). The critical distinction for **observe**(E) is whether $\Pr(E) = 0$, not whether E is of the form "$x = 3.2$".
(4) Slice will never give the wrong answer, even when all paths condition on measure-zero events. It will give the empty subdistribution, indicating that under a rejection-sampling interpretation, all samples are rejected with probability 1. The normalized posterior is then undefined because the successful-return mass is zero. Thus the user is notified, and Slice never silently gives a wrong answer.

For instance, this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3 && x >= 0.3) in false
```

results in: has raw extended-semantic masses:

$$\mu(\text{true}) = 0 \qquad \mu(\text{false}) = 0.$$

Consequently, the returned-value posterior is undefined, rather than a distribution assigning probability zero to both values.

Whereas this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3001 && x >= 0.3) in false
```

results in: has raw extended-semantic masses

$$\mu(\text{true}) = 0, \qquad \mu(\text{false}) = 0.5 \cdot 0.0001 = 0.00005, \qquad \mu(\bot) = 0.99995.$$

This is the correct behaviorOnly after normalizing over successful returns do we obtain $\Pr_{\text{post}}(\text{false}) = 1$. This agrees with rejection sampling: consider repeatedly running the program under a rejection-sampling interpretation until an observation succeeds. With probability 1, the first successful test will be **observe**$(x \leq 0.3001 \;\&\&\; x \geq 0.3)$ and the program will therefore return false with probability 1.